Rockchip BT.656 TX and BT.1120 TX Developer Guide

ID: RK-YH-YF-178

Release Version: V2.2.0

Release Date: 2025-06-18

Security Level: ☐Top-Secret ☐Secret ☐Internal ■Public

DISCLAIMER

THIS DOCUMENT IS PROVIDED “AS IS”. ROCKCHIP ELECTRONICS CO., LTD. (“ROCKCHIP”) DOES NOT PROVIDE ANY WARRANTY OF ANY KIND, EXPRESSED, IMPLIED OR OTHERWISE, WITH RESPECT TO THE ACCURACY, RELIABILITY, COMPLETENESS, MERCHANTABILITY, FITNESS FOR ANY PARTICULAR PURPOSE OR NON-INFRINGEMENT OF ANY REPRESENTATION, INFORMATION AND CONTENT IN THIS DOCUMENT. THIS DOCUMENT IS FOR REFERENCE ONLY. THIS DOCUMENT MAY BE UPDATED OR CHANGED WITHOUT ANY NOTICE AT ANY TIME DUE TO THE UPGRADES OF THE PRODUCT OR ANY OTHER REASONS.

Trademark Statement

"Rockchip", "瑞芯微", "瑞芯" shall be Rockchip’s registered trademarks and owned by Rockchip. All the other trademarks or registered trademarks mentioned in this document shall be owned by their respective owners.

All rights reserved. ©2025. Rockchip Electronics Co., Ltd.

Beyond the scope of fair use, neither any entity nor individual shall extract, copy, or distribute this document in any form in whole or in part without the written approval of Rockchip.

Rockchip Electronics Co., Ltd.

No.18 Building, No.89, Software Boulevard, TongPan Rd, Fuzhou, Fujian, PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

Preface

This document mainly introduces the debugging guide for ROCKCHIP platform BT.656 and BT.1120 interfaces.

Product Version

Chip Models	Kernel Version
RV1109/RV1126	Linux kernel version 4.19 and later
RV1103/RV1106	Linux kernel version 5.10 and later
RK356X	Linux kernel version 4.19 and later
RK3588	Linux kernel version 5.10 and later
RK3562	Linux kernel version 5.10 and later
RK3506	Linux kernel version 6.1 and later
RV1126B	Linux kernel version 6.1 and later

Intended Audience

This document (this guide) is mainly intended for:

Technical Support Engineers

Software Development Engineers

Hardware Development Engineers

Revision Record

Version	Author	Date	Modification Description
V1.0.0	Huang Jiachai	2021-04-30	Initial version
V1.1.0	Huang Jiachai	2021-07-23	Added common issues
V1.2.0	Huang Jiachai	2021-12-29	Added support for Linux 5.10 and RK3588
V1.2.1	Yan Xiaojun	2022-05-30	Fixed typos
V1.3.0	Huang Jiachai	2023-05-29	Added support for RV1103/RV1106/RK3562
V1.4.0	Ding Lingsong	2024-03-21	Added support for RK3576 and description of Blanking Interval Data
V1.5.0	Ding Lingsong	2024-07-22	Added support for RK3506 and additional hardware connection description
V2.0.0	Huang Jiachai	2025-03-12	Renamed document name
V2.1.0	Chen Chaoyi	2025-04-23	Added support for RV1126B
V2.2.0	Chen Chaoyi	2025-06-18	Added hardware connection description for RV1126B-P

Table of Contents

Rockchip BT.656 TX and BT.1120 TX Developer Guide

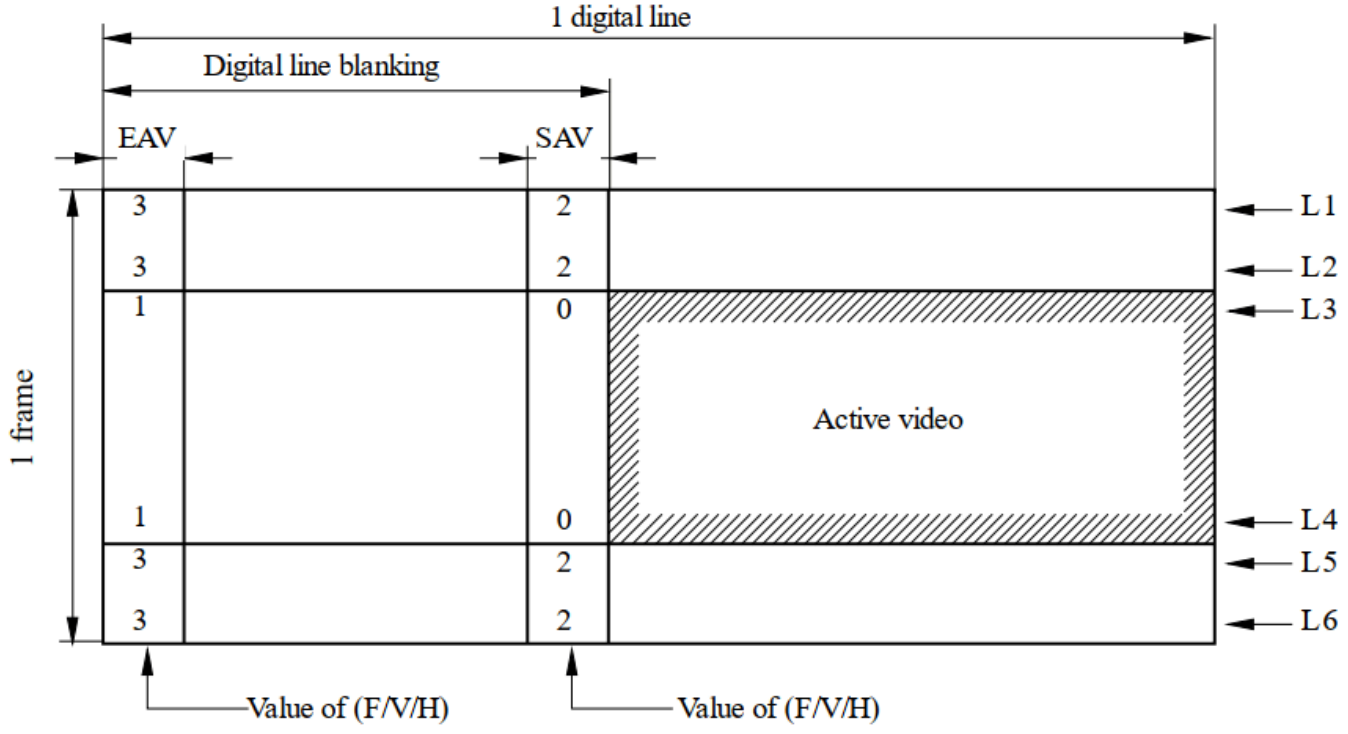
1. Concepts
   - 1.1 Progressive Scan Timing
   - 1.2 Interlaced Scan Timing
   - 1.3 Positioning Reference Code
2. RK Platform Support Status
3. Hardware Connection
   - 3.1 RV1109/RV1126/RV1103/RV1106/RK3566/RK3568/RK3588 Platforms
   - 3.2 RK3562/RK3576/RK3506/RV1126B Platforms
   - 3.3 RV1126B-P Platform
4. Software Configuration
   - 4.1 Enabling BT.656/BT.1120
   - 4.2 Timing Configuration
5. FAQ
   - 5.1 BT.656 and BT.1120 Output : Full Range or Limited Range
   - 5.2 Checking BT.656/1120 Output Configuration
   - 5.3 Are the BT.656 and BT.1120 signals output by the RK platform standard?
   - 5.4 Single Edge Trigger or Double Edge Trigger
   - 5.5 DTS Configuration Reference Files
   - 5.6 How to Drive Third-Party Conversion Chips
   - 5.7 Usage of RK628 BT.1120
   - 5.8 What is the relationship with Camera's BT.656/BT.1120
   - 5.9 Relationship between BT.656/BT.1120 and VOP
   - 5.10 Relationship between BT.656/BT.1120 and RGB
   - 5.11 Are HSYNC/VSYNC/DEN signals routed to external IOs?
   - 5.12 Data During Blanking Interval

1. Concepts

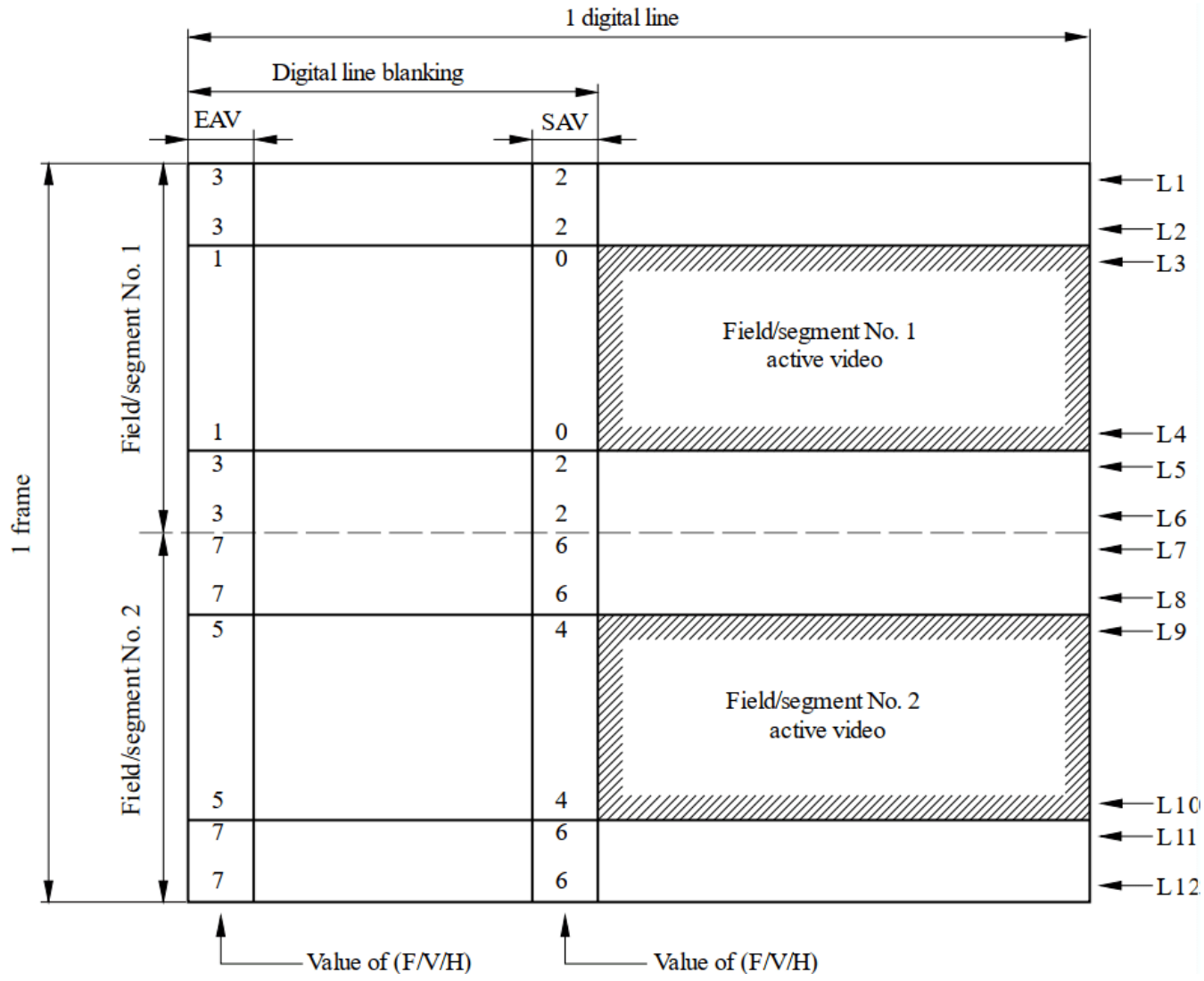
BT.656 and BT.1120 respectively define the interface protocols for SDTV and HDTV, transmitting EAV and SAV positioning reference codes as embedded synchronization signals during the blanking period, with the transmitted data format being YCbCr 4:2:2. BT.656 and BT.1120 are also referred to as video signals or YUV signals in some documents and manuals, and the bit depth of the BT.656/BT.1120 image data and positioning reference codes output by the RK platform is currently 8-bit.

Below is an introduction to some basic concepts and protocols of BT.656 and BT.1120. Detailed information can be referred to the documents "Rec. ITU-R BT.1120" and "Rec. ITU-R BT.656".

1.1 Progressive Scan Timing



1.2 Interlaced Scan Timing



1.3 Positioning Reference Code

Bit assignment for video timing reference codes

Word	Bit number									
	9 (MSB)	**8**	**7**	**6**	**5**	**4**	**3**	**2**	**1**	**0 (LSB)**
First	1	1	1	1	1	1	1	1	1	1
Second	0	0	0	0	0	0	0	0	0	0
Third	0	0	0	0	0	0	0	0	0	0
Fourth	1	F	V	H	P_3	P_2	P_1	P_0	0	0
Interlaced and segmented frame system	F = 1 during field/segment No. 2 = 0 during field/segment No. 1			V = 1 during field/segment blanking = 0 elsewhere					H = 1 in EAV = 0 in SAV	
Progressive system	F = 0			V = 1 during frame blanking = 0 elsewhere					H = 1 in EAV = 0 in SAV	

The first three words of the positioning reference code are fixed as: 0xFF, 0x00, 0x00, and the fourth word is determined by different scanning positions:

- Bit9: Fixed as 1
- Bit8(F): F=0 indicates an even field, F=1 indicates an odd field
- Bit7(V): V=0 indicates that the line contains valid video data, V=1 indicates that the line does not contain valid video data
- Bit6(H): H=0 indicates SAV, H=1 indicates EAV
- Bit[5, 2] (P3,P2,P1,P0): Calculated from Bit8 to Bit6, where:

  Bit5 = V XOR H

  Bit4 = F XOR H

  Bit3 = F XOR V

  Bit2 = F XOR V XOR H
- Bit[1, 0]: Fixed as 0, for RK platform, the bit depth is 8-bit, these 2 bits can be considered non-existent.

The following table calculates the corresponding protection bit values (P3,P2,P1,P0) for different EAV/SAV (F,V,H):

Protection bits for SAV and EAV

	SAV/EAV bit status			Protection bits					
Bit 9 (fixed)	**8 (F)**	**7 (V)**	**6 (H)**	**5 (P_3)**	**4 (P_2)**	**3 (P_1)**	**2 (P_0)**	**1 (fixed)**	**0 (fixed)**
1	0	0	0	0	0	0	0	0	0
1	0	0	1	1	1	0	1	0	0
1	0	1	0	1	0	1	1	0	0
1	0	1	1	0	1	1	0	0	0
1	1	0	0	0	1	1	1	0	0
1	1	0	1	1	0	1	0	0	0
1	1	1	0	1	1	0	0	0	0
1	1	1	1	0	0	0	1	0	0

Combining the above information, the positioning reference codes corresponding to each blanking period are:

EAV	CODE	SAV	CODE
1	0XFF 0X00 0X00 0X9D	0	0xFF 0X00 0X00 0X80
3	0XFF 0X00 0X00 0XB6	2	0xFF 0X00 0X00 0XAB
5	0XFF 0X00 0X00 0XDA	4	0xFF 0X00 0X00 0XC7
7	0XFF 0X00 0X00 0XF1	6	0xFF 0X00 0X00 0XEC

2. RK Platform Support Status

SOC Platform	Support for BT.656	Support for BT.1120	Interlaced or Progressive
RV1109/RV1126	N	Y	Only supports **progressive** scan
RV1103	Y	N	Supports **progressive** and **interlaced** scan
RV1106	Y	Y	Supports **progressive** and **interlaced** scan
RK3566/RK3568	Y	Y	Supports **progressive** and **interlaced** scan
RK3588	Y	Y	Supports **progressive** and **interlaced** scan
RK3562	Y	Y	Supports **progressive** and **interlaced** scan
RK3576	Y	Y	Supports **progressive** and **interlaced** scan
RK3506	Y	Y	Supports **progressive** and **interlaced** scan
RV1126B	Y	Y	Supports **progressive** and **interlaced** scan

3. Hardware Connection

3.1 RV1109/RV1126/RV1103/RV1106/RK3566/RK3568/RK3588 Platforms

BT.656 and BT.1120 support the following three hardware connections, and software should adapt the bus_format in the DTS file or the corresponding conversion chip driver according to the connection method.

SOC TX pin	Clock	D15	D14	D13	D12	D11	D10	D9	D8	D7	D6	D5	D4	D3	D2	D1	D0	The bus_format Software Configuration
BT.656 Connection	Clock	—	—	—	—	—	—	—	—	D7	D6	D5	D4	D3	D2	D1	D0	MEDIA_BUS_FMT_UYVY8_2X8
BT.1120 Connection 1	Clock	Y7	Y6	Y5	Y4	Y3	Y2	Y1	YO	C7	C6	C5	C4	C3	C2	C1	CO	MEDIA_BUS_FMT_YUYV8_1X16
BT.1120 Connection 2	Clock	C7	C6	C5	C4	C3	C2	C1	CO	Y7	Y6	Y5	Y4	Y3	Y2	Y1	Y0	MEDIA_BUS_FMT_UYVY8_1X16

3.2 RK3562/RK3576/RK3506/RV1126B Platforms

The BT.656 interface simultaneously outputs image data to the M0 and M1 pin groups, which can be configured as a single display or dual-screen display mode according to the application scenario.

Component Name	Pin Name	BT1120	BT656
Bus Format	×	MEDIA_BUS_FMT_YUYV8_1X16/ MEDIA_BUS_FMT_UYVY8_1X16	MEDIA_BUS_FMT_UYVY8_2X8
DCLK	VO_LCDC_CLK	DCLK(RS)	DCLK(RS)
R7_D23	VO_LCDC_D23	√	√ (D7_m1)
R6_D22	VO_LCDC_D22	√	√ (D6_m1)
R5_D21	VO_LCDC_D21	√	√ (D5_m1)
R4_D20	VO_LCDC_D20	√	√ (D4_m1)
R3_D19	VO_LCDC_D19	√	√ (D3_m1)
R2_D18	VO_LCDC_D18	×	×
R1_D17	VO_LCDC_D17	×	×
R0_D16	VO_LCDC_D16	×	×
G7_D15	VO_LCDC_D15	√	√ (D2_m1)
G6_D14	VO_LCDC_D14	√	√ (D1_m1)
G5_D13	VO_LCDC_D13	√	√ (D0_m1)
G4_D12	VO_LCDC_D12	√	√ (D7_m0)
G3_D11	VO_LCDC_D11	√	√ (D6_m0)
G2_D10	VO_LCDC_D10	√	√ (D5_m0)
G1_D9	VO_LCDC_D9	×	×
G0_D8	VO_LCDC_D8	×	×
B7_D7	VO_LCDC_D7	√	√ (D4_m0)
B6_D6	VO_LCDC_D6	√	√ (D3_m0)
B5_D5	VO_LCDC_D5	√	√ (D2_m0)
B4_D4	VO_LCDC_D4	√	√ (D1_m0)
B3_D3	VO_LCDC_D3	√	√ (D0_m0)
B2_D2	VO_LCDC_D2	×	×
B1_D1	VO_LCDC_D1	×	×
B0_D0	VO_LCDC_D0	×	×

3.3 RV1126B-P Platform

For the RV1126B-P platform, to maintain consistency with the hardware connection methods of the earlier RV1126 platform, the concept of data map has been introduced. By setting different data maps, data can be output on different hardware pins.

The complete definition of data map can be found in `include/dt-bindings/display/rockchip_vop.h`, as follows:

```
/* { 8'd0, BT1120[15:0] } */
#define ROCKCHIP_BT1120_DATA_MAP_DATA_PADLO     0
/* { BT1120[15:0], 8'd0 }*/
#define ROCKCHIP_BT1120_DATA_MAP_DATA_PADHI     1
/*
 * { BT1120[15:11], 3'd0, BT1120[10:5], 2'd0, BT1120[4:0], 3'b0 }
 * That is:
 *     LCDC_D23 -> BT1120_D15
 *     LCDC_D22 -> BT1120_D14
 *     LCDC_D21 -> BT1120_D13
 *     LCDC_D20 -> BT1120_D12
 *     LCDC_D19 -> BT1120_D11
 *     LCDC_D18 -> N/A
 *     LCDC_D17 -> N/A
 *     LCDC_D16 -> N/A
 *     LCDC_D15 -> BT1120_D10
 *     LCDC_D14 -> BT1120_D9
 *     LCDC_D13 -> BT1120_D8
 *     LCDC_D12 -> BT1120_D7
 *     LCDC_D11 -> BT1120_D6
 *     LCDC_D10 -> BT1120_D5
 *     LCDC_D9  -> N/A
 *     LCDC_D8  -> N/A
 *     LCDC_D7  -> BT1120_D4
 *     LCDC_D6  -> BT1120_D3
 *     LCDC_D5  -> BT1120_D2
 *     LCDC_D4  -> BT1120_D1
 *     LCDC_D3  -> BT1120_D0
 *     LCDC_D2  -> N/A
 *     LCDC_D1  -> N/A
 *     LCDC_D0  -> N/A
 */
#define ROCKCHIP_BT1120_DATA_MAP_DATA_WITH_DUMMY_0  3
/* { 16'd0, BT656[7:0] }*/
#define ROCKCHIP_BT656_DATA_MAP_PADLO           0
/* { 8'd0, BT656[7:0], 8'd0 }*/
#define ROCKCHIP_BT656_DATA_MAP_DUMMY_HI_LO     1
/* { BT656[7:0], 16'd0 }*/
#define ROCKCHIP_BT656_DATA_MAP_PADHI           2
/*
 * { BT656[7:3], 3'd0, BT656[2:0], BT656[7:5], 2'd0, BT656[4:0], 3'd0 }
 * That is:
 *     LCDC_D23 -> BT656_D7_M1
 *     LCDC_D22 -> BT656_D6_M1
 *     LCDC_D21 -> BT656_D5_M1
 *     LCDC_D20 -> BT656_D4_M1
 *     LCDC_D19 -> BT656_D3_M1
 *     LCDC_D18 -> N/A
 *     LCDC_D17 -> N/A
 *     LCDC_D16 -> N/A
 *     LCDC_D15 -> BT656_D2_M1
 *     LCDC_D14 -> BT656_D1_M1
 *     LCDC_D13 -> BT656_D0_M1
```

```
 *    LCDC_D12 -> BT656_D7_M0
 *    LCDC_D11 -> BT656_D6_M0
 *    LCDC_D10 -> BT656_D5_M0
 *    LCDC_D9  -> N/A
 *    LCDC_D8  -> N/A
 *    LCDC_D7  -> BT656_D4_M0
 *    LCDC_D6  -> BT656_D3_M0
 *    LCDC_D5  -> BT656_D2_M0
 *    LCDC_D4  -> BT656_D1_M0
 *    LCDC_D3  -> BT656_D0_M0
 *    LCDC_D2  -> N/A
 *    LCDC_D1  -> N/A
 *    LCDC_D0  -> N/A
 */
#define ROCKCHIP_BT656_DATA_MAP_DATA_WITH_DUMMY_0   3
```

For RV1126B, the default hardware connection methods are as follows, corresponding to `ROCKCHIP_BT1120_DATA_MAP_DATA_WITH_DUMMY_0` and `ROCKCHIP_BT656_DATA_MAP_DATA_WITH_DUMMY_0`.

Component Name	Pin Name	BT1120	BT656
Bus Format	×	MEDIA_BUS_FMT_YUYV8_1X16/ MEDIA_BUS_FMT_UYVY8_1X16	MEDIA_BUS_FMT_UYVY8_2X8
DCLK	VO_LCDC_CLK	DCLK(RS)	DCLK(RS)
R7_D23	VO_LCDC_D23	√	√ (D7_m1)
R6_D22	VO_LCDC_D22	√	√ (D6_m1)
R5_D21	VO_LCDC_D21	√	√ (D5_m1)
R4_D20	VO_LCDC_D20	√	√ (D4_m1)
R3_D19	VO_LCDC_D19	√	√ (D3_m1)
R2_D18	VO_LCDC_D18	×	×
R1_D17	VO_LCDC_D17	×	×
R0_D16	VO_LCDC_D16	×	×
G7_D15	VO_LCDC_D15	√	√ (D2_m1)
G6_D14	VO_LCDC_D14	√	√ (D1_m1)
G5_D13	VO_LCDC_D13	√	√ (D0_m1)
G4_D12	VO_LCDC_D12	√	√ (D7_m0)
G3_D11	VO_LCDC_D11	√	√ (D6_m0)
G2_D10	VO_LCDC_D10	√	√ (D5_m0)
G1_D9	VO_LCDC_D9	×	×
G0_D8	VO_LCDC_D8	×	×
B7_D7	VO_LCDC_D7	√	√ (D4_m0)
B6_D6	VO_LCDC_D6	√	√ (D3_m0)
B5_D5	VO_LCDC_D5	√	√ (D2_m0)
B4_D4	VO_LCDC_D4	√	√ (D1_m0)
B3_D3	VO_LCDC_D3	√	√ (D0_m0)
B2_D2	VO_LCDC_D2	×	×
B1_D1	VO_LCDC_D1	×	×
B0_D0	VO_LCDC_D0	×	×

For RV1126B-P, to maintain consistency with the earlier RV1126 hardware connection methods, the default connection methods are `ROCKCHIP_BT1120_DATA_MAP_DATA_PADLO` (for BT1120) or `ROCKCHIP_BT656_DATA_MAP_PADLO` (for BT656), as shown below:

SOC TX pin	Clock	D15	D14	D13	D12	D11	D10	D9	D8	D7	D6	D5	D4	D3	D2	D1	D0	The bus_format Software Configuration
BT.656 Connection	Clock	—	—	—	—	—	—	—	—	D7	D6	D5	D4	D3	D2	D1	D0	MEDIA_BUS_FMT_UYVY8_2X8
BT.1120 Connection 1	Clock	Y7	Y6	Y5	Y4	Y3	Y2	Y1	YO	C7	C6	C5	C4	C3	C2	C1	CO	MEDIA_BUS_FMT_YUYV8_1X16
BT.1120 Connection 2	Clock	C7	C6	C5	C4	C3	C2	C1	CO	Y7	Y6	Y5	Y4	Y3	Y2	Y1	Y0	MEDIA_BUS_FMT_UYVY8_1X16

Generally, RV1126B and RV1126B-P have predefined default connection methods, and no additional settings are required. However, if modification is indeed necessary, it can be set in the dts.

The following example shows the configuration of RV1126B using the earlier RV1126 hardware connection method (for RV1126B-P, this configuration is already set by default):

```
&rgb {
    /*
     * RV1126 compatible pin output mode 0.
     *
     * BT1120: ROCKCHIP_BT1120_DATA_MAP_DATA_PADLO
     * BT656: ROCKCHIP_BT656_DATA_MAP_PADLO
     */
    rockchip,data-map-mode = <0>;
};
```

4. Software Configuration

4.1 Enabling BT.656/BT.1120

1. For devices that do not require software drivers on the transmitting end (i.e., no need to register DRM connector), this can be configured in the panel node of the dts file:

```
panel {
    ...
    bus-format = MEDIA_BUS_FMT_YUYV8_1X16; // or MEDIA_BUS_FMT_UYVY8_1X16
    ...
}
```

2. For devices that require software drivers on the transmitting end (i.e., need to register DRM connector), in addition to referring to point 1 for adaptation in dts, it is also possible to set it in the corresponding connector driver's drm_connector_helper_funcs -> get_modes function. You can refer to the implementation in drivers/gpu/drm/bridge/sii902x.c:

```
static int sii902x_get_modes(struct drm_connector *connector)
{
    u32 bus_format = MEDIA_BUS_FMT_YUYV8_1X16;//depend on hardware
    ...
    drm_display_info_set_bus_formats(&connector->display_info, &bus_format, 1);
    ...
}
```

By configuring bus_format as described in points 1/2, the VOP driver will enable BT.656/BT.1120 and simultaneously configure the corresponding pin mapping relationships.

4.2 Timing Configuration

There are three methods for timing configuration:

1. Configuration in DTS

For products that support fixed resolutions, the corresponding timing can be configured in the DTS panel:

- P System Timing

```
timing_1080p: timing-1080p {
    clock-frequency = <148500000>;
    hactive = <1920>;
    vactive = <1080>;
    hback-porch = <100>;
    hfront-porch = <200>;
    vback-porch = <10>;
    vfront-porch = <10>;
    hsync-len = <20>;
    vsync-len = <20>;
    hsync-active = <0>;
    vsync-active = <0>;
    de-active = <0>;
    pixelclk-active = <0>;
};
```

- I System Timing

```
timing_ntsc: timing-ntsc {
    clock-frequency = <13500000>;
    hactive = <720>;
    vactive = <480>;
    hback-porch = <43>;
    hfront-porch = <33>;
    vback-porch = <36>;
    vfront-porch = <3>;
    hsync-len = <62>;
    vsync-len = <6>;
    hsync-active = <0>;
    vsync-active = <0>;
```

```
    de-active = <0>;
    pixelclk-active = <0>;
    interlaced;
    doubleclk; //only NTSC(480i60) mode and PAL(576i50) mode need this property
};
```

2. Reading EDID

For display devices that support multiple resolution inputs and have EDID information, you can refer to the sii902x driver to read the EDID information through DDC/I2C to obtain the supported resolutions of the display device:

```
//dts
&i2c3 {
    clock-frequency = <400000>;
    status = "okay";
    sii9022: sii9022@39 {
        compatible = "sil,sii9022";
        reg = <0x39>;
        pinctrl-names = "default";
        ...
        ports {
            #address-cells = <1>;
            #size-cells = <0>;
            port@0 {
                reg = <0>;
                sii9022_in_rgb: endpoint {
                    remote-endpoint = <&rgb_out_sii9022>;
                };
            };
        };
    };
};

&rgb {
    status = "okay";
    ...
    ports {
        port@1 {
            reg = <1>;
            #address-cells = <1>;
            #size-cells = <0>;

            rgb_out_sii9022: endpoint@0 {
                reg = <0>;
                remote-endpoint = <&sii9022_in_rgb>;
            };
        };
    };
};

//drivers/gpu/drm/bridge/sii902x.c
static int sii902x_probe(struct i2c_client *client, const struct i2c_device_id *id)
```

```
{
    ...
    i2c_set_clientdata(client, sii902x);
    sii902x->i2cmux =
    i2c_mux_alloc(client->adapter, dev, 1, 0, I2C_MUX_GATE, sii902x_i2c_bypass_select, sii902x_i2c_bypass_deselect);
    if (!sii902x->i2cmux)
        return -ENOMEM;
    sii902x->i2cmux->priv = sii902x;
    return i2c_mux_add_adapter(sii902x->i2cmux, 0, 0, 0);
    ...
}

static int sii902x_get_modes(struct drm_connector *connector)
{
    struct sii902x *sii902x = connector_to_sii902x(connector);

    edid = drm_get_edid(connector, sii902x->i2cmux->adapter[0]);
    drm_connector_update_edid_property(connector, edid);
    if (edid) {
        if (drm_detect_hdmi_monitor(edid))
            output_mode = SII902X_SYS_CTRL_OUTPUT_HDMI;
        num = drm_add_edid_modes(connector, edid);
        kfree(edid);
    }
}
```

3. Writing in connector driver

This is generally done during debugging for convenience or when there is no I2C/DDC channel to read EDID information but multiple resolutions need to be supported. The corresponding resolutions can be directly written in the connector driver, which can be referred to in the implementation of sii902x.c driver:

```
static int sii902x_get_modes(struct drm_connector *connector)
{
    struct sii902x *sii902x = connector_to_sii902x(connector);

    ...
    for (i = 0; i < ARRAY_SIZE(sii902x_default_modes); i++) {
        const struct drm_display_mode *ptr = &sii902x_default_modes[i];

        mode = drm_mode_duplicate(connector->dev, ptr);
        if (mode) {
            if (!i)
                mode->type = DRM_MODE_TYPE_PREFERRED;
            drm_mode_probed_add(connector, mode);
            ret++;
        }
    }
    ...
}
```

5. FAQ

5.1 BT.656 and BT.1120 Output : Full Range or Limited Range

Answer: Limited range, meaning the effective image data range is [16, 235], and only the positioning reference code may exhibit 0xFF or 0x00 data.

5.2 Checking BT.656/1120 Output Configuration

Answer: By using `cat /sys/kernel/debug/dri/0/summary`, you can view the `bus_format` value under the corresponding VOP/VP node, which corresponds to the table in the Section 3 of this document:

```
cat /sys/kernel/debug/dri/0/summary
  Video Port0: ACTIVE
  ...
  bus_format[2025]: YUV8_1X24
  ...
```

5.3 Are the BT.656 and BT.1120 signals output by the RK platform standard?

Answer: Yes, designed based on the standards of Rec. ITU-R BT.656 and Rec. ITU-R BT.1120.

5.4 Single Edge Trigger or Double Edge Trigger

Answer: Single edge trigger is the default configuration, with the clock's rising edge in the middle of the data. If the receiver side prefers the falling edge to be in the middle of the data, the `pixelclk-active` can be set to 1 in the `dts`.

5.5 DTS Configuration Reference Files

Answer: The following two configuration files can be referenced:

BT.656: arch/arm64/boot/dts/rockchip/rk3568-evb6-ddr3-v10-rk630-bt656-to-cvbs.dts

BT.1120: arch/arm64/boot/dts/rockchip/rk3568-evb2-lp4x-v10-bt1120-to-hdmi.dts

5.6 How to Drive Third-Party Conversion Chips

Answer: There are two scenarios to consider:

- If the third-party conversion chip does not require separate configuration of registers and can function normally with just power supply, in this case, only the corresponding GPIO and power supply need to be configured in the panel node of the dts file to ensure the conversion chip has a normal power supply. No additional drivers need to be written, as the rockchip_rgb.c file will handle the registration of the encoder and connector with the DRM driver framework.
- If the third-party conversion chip requires separate configuration of registers, in addition to correctly configuring the GPIO and power supply in the dts to ensure the conversion chip has a normal power supply, a driver corresponding to the conversion chip must also be written. In this case, rockchip_rgb.c will handle the registration of the encoder with the DRM driver framework, and the driver for the conversion chip will handle the registration of the connector with the DRM driver framework, and they will be bridged together through a DRM bridge. For specifics, you can refer to the implementation in the kernel code: drivers/gpu/drm/bridge/sii902x.c.

5.7 Usage of RK628 BT.1120

Answer: For the usage of RK628 BT.1120, please refer to the document "Rockchip_DRM_RK628_Porting_Guide_CN" for details.

5.8 What is the relationship with Camera's BT.656/BT.1120

Answer: This section introduces BT.656 TX and BT.1120 TX, which are parallel output interfaces, while Camera corresponds to BT.656 RX and BT.1120 RX, which are parallel input interfaces. Both are consistent in terms of protocol. For development documentation related to BT.656 RX and BT.1120 RX, please obtain it from our company's FAE window/ISP department.

5.9 Relationship between BT.656/BT.1120 and VOP

Answer: BT.656 and BT.1120 are parallel output interfaces for embedded synchronous signals. VOP outputs the data that has been synthesized from multiple layers [multiple buffers] through BT.656/BT.1120.

5.10 Relationship between BT.656/BT.1120 and RGB

Answer: BT.656 and BT.1120 are parallel output interfaces for embedded synchronous signals, while RGB is a parallel output interface with independent synchronization signals [HSYNC/VSYNC/DEN]. They are all independent display interfaces and do not have a direct relationship in the display pathway, although they may have multiplexing relationships at the IO level.

5.11 Are HSYNC/VSYNC/DEN signals routed to external IOs?

Answer: It varies by platform:

- RV1126/RV1109/RK3562/RK3576

  HSYNC/VSYNC/DEN signals are all routed to external IOs, but Data lags behind HSYNC/VSYNC/DEN by one clock cycle.

- RV1126B

  HSYNC/VSYNC/DEN signals are all routed to external IOs. For BT1120 output, Data is synchronized with HSYNC/VSYNC/DEN; for BT656, Data lags behind HSYNC/VSYNC/DEN by 2 clock cycles.

- RK3566/RK3568/RK3588

  HSYNC/VSYNC/DEN signals are not routed to external IOs.

- RV1103/RV1106/RK3506

  HSYNC/VSYNC/DEN signals are all routed to external IOs, and Data is synchronized with HSYNC/VSYNC/DEN.

5.12 Data During Blanking Interval

Answer: Distinguished by platform:

- For RV1126/RK3566/RK3568/RK3588

  The BT.656 blanking interval data is 0x80 0x10 0x80 0x10;

  The BT.1120 Y/UV channel blanking interval data is 0x80 0x10 0x80 0x10, which differs from the protocol. For some receivers sensitive to blanking interval data, display anomalies may occur.

- For other platforms

  The BT.656 blanking interval data is 0x80 0x10 0x80 0x10;

  The BT.1120 Y channel blanking interval data is 0x80 while the UV channel blanking interval data is 0x10.